Relatório Técnico – Sistema Acadêmico FACENS

Seção 1 – Apresentação do Sistema

O Sistema Acadêmico FACENS é um programa em C, executado no terminal, que registra alunos, disciplinas e notas, calcula a média de cada aluno e informa se ele está aprovado ou reprovado. Os dados são gravados em arquivo binário e reaparecem na próxima execução.

Problema que o sistema resolve

Controlar notas e disciplinas em papel ou em planilhas soltas gera perda de informação, cálculos de média feitos à mão e dificuldade para consultar um aluno específico. O sistema centraliza esses dados e automatiza as tarefas repetitivas:

- cadastro de alunos (nome, RA, disciplina e quatro notas);
- cadastro de disciplinas (nome, código e carga horária), sem permitir dois códigos iguais;
- cálculo automático da média e do status (aprovado com média maior ou igual a 5,0);
- consulta de aluno pelo RA e de disciplina pelo código;
- atualização de uma nota com recálculo da média;
- armazenamento permanente em arquivo, sem depender de um banco de dados externo.

Usuários do sistema

O usuário principal é quem administra os dados acadêmicos: a secretaria, a coordenação de curso e os professores que lançam notas. O sistema usa menus numerados, então não exige conhecimento técnico para operar.

Motivação da escolha do tema

O domínio acadêmico é familiar a todos os integrantes do grupo e naturalmente exige os conceitos da disciplina: duas estruturas de dados relacionadas (aluno e disciplina), coleções que crescem durante a execução (vetores dinâmicos), buscas por chave (RA e código), cálculos sobre os dados (média) e persistência em arquivo binário. Assim, cada requisito do projeto aparece como uma necessidade real do sistema, e não como um exercício isolado.

Seção 2 – Modelagem

O sistema usa duas structs: `Disciplina` e `Alunos`. A segunda contém a primeira. Os dados de cada struct ficam em um vetor dinâmico global (`disciplinas` e `alunos`), acompanhado de um contador (`quant_disciplinas` e `quant`).

Struct Disciplina

Campo	Tipo	Função
nome	char[50]	Nome da disciplina, exibido em listagens e consultas

Campo	Tipo	Função
codigo	int	Identificador único; é a chave de busca e impede cadastro duplicado
carga_horaria	int	Carga horária em horas, usada na consulta de carga horária

Struct Alunos

Campo	Tipo	Função
nome	char[100]	Nome do aluno, exibido em listagens e consultas
notas	float[4]	As quatro notas (AC 1 a AC 4) que compõem a média
ra	int	Registro do aluno; é a chave de busca por aluno
disciplina	Disciplina	Disciplina cursada, guardada dentro do próprio registro do aluno
media	float	Média aritmética das quatro notas, base do status de aprovação

Relacionamento entre as estruturas

A relação é de composição: `Alunos` tem um campo do tipo `Disciplina`, então cada aluno carrega uma cópia dos dados da disciplina que cursa. Os dois vetores globais são independentes. Isso simplifica a gravação em arquivo, porque cada aluno é um bloco completo de bytes. Em contrapartida, a disciplina informada no cadastro do aluno não é conferida contra o vetor `disciplinas`, e alterar uma disciplina não atualiza os alunos já cadastrados. Ligar os dois pelo `codigo` seria uma evolução natural.

Por que cada informação é importante?

- **ra e codigo:** são as chaves de busca. Sem um valor que identifique cada registro, as funções `buscar_aluno` e `buscar_disciplina` não teriam como localizar o item certo, e o código ainda evita disciplinas duplicadas.
- **nome:** permite ao usuário reconhecer o registro nas listagens e consultas.
- **notas:** são a matéria-prima do cálculo. Guardar as quatro notas, e não só a média, permite corrigir uma nota depois e recalcular.
- **media:** evita refazer a conta a cada listagem e define o status (aprovado com 5,0 ou mais).
- **carga_horaria:** é a informação que a consulta de carga horária devolve e descreve o peso da disciplina no curso.
- **disciplina (dentro de Alunos):** vincula as notas a uma disciplina, de modo que a média tenha contexto.
- **quant e quant_disciplinas:** não pertencem a uma struct, mas controlam o tamanho dos vetores dinâmicos, os laços de percurso e o que será gravado no arquivo.

Seção 3 – Planejamento da Implementação

O sistema foi organizado em três níveis de menu (principal, alunos e disciplinas), cada um chamando funções pequenas e com uma única responsabilidade.

Principais funcionalidades

Menu	Funcionalidade	Função no código
Alunos	Cadastrar aluno (dados, disciplina e 4 notas, com cálculo da média)	`cadastrarAluno`, `calcular_media`
Alunos	Listar alunos com média e status	`listar_alunos`
Alunos	Consultar aluno pelo RA	`consultar_alunc buscar_aluno` ,
Alunos	Atualizar uma nota e recalcular a média	`modificar_nota_direta`
Disciplinas	Cadastrar disciplina (rejeita código repetido)	`cadastrarDisciplina`
Disciplinas	Listar disciplinas	`listar_disciplinas`
Disciplinas	Buscar disciplina pelo código	`buscar_disciplina`
Disciplinas	Consultar carga horária	`consultar_carga_horaria`
Sistema	Carregar dados ao iniciar e salvar ao encerrar	`carregar_dados salvar_dados` ,

Decisões de projeto

- **Vetores dinâmicos em vez de vetores fixos:** o sistema não impõe limite de alunos nem de disciplinas e usa só a memória necessária.
- **Crescimento de um em um com `realloc`:** cada novo cadastro amplia o vetor em uma posição. É simples de entender e o custo não pesa para o volume esperado.
- **Funções de busca que devolvem ponteiro:** `buscar_aluno` e `buscar_disciplina` retornam o endereço do registro (ou `NULL`), o que permite ler e alterar o item sem copiá-lo.
- **Verificação de código duplicado:** a disciplina só é gravada se o código ainda não existir.
- **Ponteiro temporário no `realloc`:** o retorno vai primeiro para `cont` ou `temp`, de modo que uma falha de alocação não faça o programa perder o vetor original.
- **Menus separados por assunto:** `sistema_academico`, `menu_alunos` e `menu_disciplinas` deixam o fluxo fácil de seguir e de ampliar.
- **Persistência em um único arquivo binário:** `dados_sistema.bin` guarda contadores e vetores. Os dados são lidos no início de `main` e gravados no fim.
- **Memória liberada ao encerrar:** `main` chama `free` nos dois vetores.

Divisão de responsabilidades entre os integrantes

Integrante	Responsabilidade
Luiz	Structs, cadastro e listagem de alunos, cálculo da média
Gabriel	Cadastro, listagem e busca de disciplinas

Integrante	Responsabilidade
Frederico	Persistência em arquivo binário (`salvar_dados` `carregar_dados`)
Dyulian	Menus, atualização de nota por ponteiro duplo, bateria de testes e relatório

Seção 4 – Registro de uso de IA

O grupo usou duas ferramentas de IA como apoio e revisou tudo o que recebeu antes de colocar no projeto. Os pedidos eram feitos em linguagem natural, descrevendo a funcionalidade desejada (por exemplo, "cadastrar alunos usando vetor dinâmico e realloc"). A IA devolvia um trecho de código ou uma explicação, e o grupo testava, corrigia e adaptava o resultado ao restante do sistema.

Ferramentas utilizadas: ChatGPT, que gerou a maior parte do código, e Claude (Anthropic), usado para contribuir com partes específicas, como a explicação do ponteiro para ponteiro, a revisão de trechos e a elaboração do relatório e da bateria de testes.

Tabela de uso

Situação	O que foi solicitado à IA	O resultado foi utilizado?	Modificações realizadas pelo grupo
Exemplo: cadastro de produtos	Implementar cadastro de produtos	Parcialmente	Foram adicionadas validações e correções
Estrutura inicial do programa (ChatGPT)	Criar as structs de aluno e disciplina, os vetores dinâmicos globais e os menus em C	Parcialmente	A disciplina foi embutida na struct `Alunos`, os menus foram separados em principal, alunos e disciplinas e os nomes das funções foram padronizados
Cadastro de alunos com `realloc` (ChatGPT)	Implementar o cadastro de alunos em vetor dinâmico, calculando a média das 4 notas	Sim, com ajustes	Uso de ponteiro temporário no `realloc`, teste de `NULL` e exibição da média e do status ao final do cadastro
Funções de busca por RA e por código (ChatGPT)	Criar funções que procuram um aluno ou uma disciplina no vetor e retornam ponteiro	Sim	Percurso reescrito com aritmética de ponteiros, `(a + i)->ra`, e uso do retorno `NULL` nos menus
Cadastro de disciplina sem código repetido (ChatGPT)	Cadastrar disciplinas no vetor dinâmico	Parcialmente	Inclusão da busca por código antes de pedir a carga horária, para recusar duplicados
Atualização de	Explicar o ponteiro duplo	Parcialmente	Chamada integrada ao menu com

nota com	e escrever uma função		`buscar_aluno` e `&a`, validação do

Situação	O que foi solicitado à IA	O resultado foi utilizado?	Modificações realizadas pelo grupo
ponteiro para ponteiro (Claude)	que altera uma nota recebendo `Alunos **`		número da nota (1 a 4) e recálculo da média
Gravação e leitura em arquivo binário (ChatGPT)	Salvar e carregar contadores e vetores em `dados_sistema.bin`	Parcialmente	Tratamento da primeira execução sem arquivo, teste do `malloc` na leitura, `free` no fim de `main` e exibição do tamanho com `fseek` e `ftell`
Relatório técnico e bateria de testes (Claude)	Organizar o relatório nas seções pedidas e propor casos de teste com base no código	Parcialmente	O grupo conferiu o texto com o código, completou as partes pessoais e executou os testes para confirmar os resultados

Sugestões da IA que precisaram ser alteradas ou corrigidas

Estas foram as principais sugestões das IAs que o grupo precisou alterar ou corrigir:

Situação 1: realocação de memória no cadastro

- **O que a IA gerou inicialmente:** um `realloc` atribuído direto ao ponteiro global do vetor, sem conferir se a alocação deu certo.
- **O que foi alterado pelo grupo:** o retorno passou a ir para um ponteiro temporário (`cont` e `temp`), testado contra `NULL` antes de atualizar `alunos` e `disciplinas`.
- **Por que a alteração foi necessária:** se o `realloc` falhasse, o ponteiro global viraria `NULL` e o vetor com todos os cadastros seria perdido, além de vazar memória.

Situação 2: disciplina com código repetido

- **O que a IA gerou inicialmente:** um cadastro de disciplina que lia os dados e gravava no vetor sem verificar se o código já existia.
- **O que foi alterado pelo grupo:** foi incluída a chamada a `buscar_disciplina` logo depois da leitura do código, antes de pedir a carga horária, com mensagem de erro e retorno da função se o código já existir.
- **Por que a alteração foi necessária:** o código é a chave de busca. Com dois códigos iguais, a consulta devolveria sempre a primeira disciplina e a segunda ficaria inacessível.

Situação 3: atualização de nota com ponteiro para ponteiro

- **O que a IA gerou inicialmente:** uma função que recebia `Alunos **` e trocava o valor de uma nota, sem indicar como chamá-la a partir do menu.
- **O que foi alterado pelo grupo:** a chamada foi integrada à opção 4 do menu, usando `buscar_aluno` para localizar o aluno e `&a` como argumento. Foi acrescentada a validação

do número da nota (1 a 4) e a chamada a `calcular_media` dentro da função.

- **Por que a alteração foi necessária:** sem o recálculo, a média e o status mostrados ficariam desatualizados depois da troca da nota. Sem a validação, um número fora de 1 a 4 acessaria posição inexistente do vetor `notas`.

Situação 4: leitura do arquivo binário

- **O que a IA gerou inicialmente:** funções de salvar e carregar que gravavam e liam os vetores, mas partiam do princípio de que o arquivo sempre existia.
- **O que foi alterado pelo grupo:** `carregar_dados` passou a retornar sem erro quando `fopen` devolve `NULL`, o `malloc` da leitura passou a ser testado, e `main` passou a liberar os vetores com `free`. Em `salvar_dados`, o grupo acrescentou `fseek` e `ftell` para mostrar o tamanho gravado.
- **Por que a alteração foi necessária:** na primeira execução o arquivo ainda não existe, e o programa não pode falhar nem travar. O teste do `malloc` evita ler dados em ponteiro nulo, e liberar a memória é parte do uso correto da alocação dinâmica.

Seção 5 – Ponteiros e Alocação Dinâmica

Onde foram utilizados ponteiros e por que eram necessários

- **Vetores globais `alunos` e `disciplinas`:** são ponteiros (`Alunos *` e `Disciplina *`) que começam em `NULL` e passam a apontar para blocos alocados em tempo de execução. Como o número de registros não é conhecido de antemão, um vetor de tamanho fixo não serviria.
- **Funções de busca (`buscar_aluno`, `buscar_disciplina`):** recebem o ponteiro inicial do vetor e devolvem o endereço do registro encontrado, ou `NULL` se não existir. Devolver o endereço evita copiar a struct e permite alterar o original. O valor `NULL` indica "não encontrado".
- **Aritmética de ponteiros:** o acesso `(a + i)->ra` percorre o vetor deslocando o ponteiro `i` posições. É equivalente a `a[i].ra`, mas deixa explícito que o vetor é um bloco contíguo de memória.
- **`calcular_media(Alunos *a)`:** recebe o endereço do aluno para gravar a média diretamente no registro. Com passagem por valor, o cálculo ficaria numa cópia e se perderia.
- **`cadastrarAluno`:** usa `Alunos *a = &alunos[quant]` para preencher o novo registro com `scanf`, passando `&a->ra`, `&a->notas[i]` e outros endereços.
- **`cadastrarDisciplina`:** copia a struct preenchida para a posição nova do vetor com `*d = nova`, em que `d` aponta para `disciplinas + quant_disciplinas`.
- **Menus e consultas:** `consultar_aluno`, `consultar_carga_horaria` e o case de busca em `menu_disciplinas` guardam o retorno das buscas em um ponteiro e testam se é `NULL` antes de usá-lo.
- **Arquivo:** `FILE *arq` é o ponteiro que representa o arquivo aberto em `salvar_dados` e `carregar_dados`.

Onde foi utilizado ponteiro para ponteiro

A função `modificar_nota_direta(Alunos **ptr_para_ponteiro, int indice_nota, float nova_nota)` recebe o endereço de um ponteiro para `Alunos`. Dentro dela, `(*ptr_para_ponteiro)->notas[indice_nota]` primeiro desreferencia o ponteiro duplo, para chegar ao ponteiro do aluno, e depois acessa a nota. Em seguida, `calcular_media(*ptr_para_ponteiro)` recalcula a média.

No menu de alunos (opção 4), o aluno é localizado com `buscar_aluno` e a função é chamada com `modificar_nota_direta(&a, num_nota - 1, nova_nota)`. Aqui, `&a` é o endereço da variável local `a`. Para o efeito pedido, um ponteiro simples bastaria. O ponteiro duplo foi usado para demonstrar o conceito, e passaria a ser indispensável se a função precisasse trocar para onde `a` aponta, por exemplo ao realocar o vetor dentro dela.

Onde ocorreu alocação dinâmica e qual estratégia foi utilizada

Local	Função usada	Estratégia
`cadastrarAluno`	`realloc`	Cresce o vetor `alunos` em uma posição a cada cadastro
`cadastrarDisciplina`	`realloc`	Cresce o vetor `disciplinas` em uma posição a cada cadastro
`carregar_dados`	`malloc`	Aloca o vetor com exatamente `quant` (ou `quant_disciplinas`) elementos lidos do arquivo
`main`	`free`	Libera os dois vetores antes de encerrar

A estratégia é de crescimento linear sob demanda: o vetor só ocupa a memória dos registros existentes. Todo retorno de `malloc` e `realloc` é testado contra `NULL`, e o programa avisa o erro em vez de continuar com ponteiro inválido.

Onde ocorreu realocação da memória

A realocação acontece em dois pontos: na linha `realloc(alunos, (quant + 1) * sizeof(Alunos))` de `cadastrarAluno` e na linha `realloc(disciplinas, (quant_disciplinas + 1) * sizeof(Disciplina))` de `cadastrarDisciplina`. Na primeira chamada o ponteiro vale `NULL`, e `realloc` se comporta como `malloc`. Nas seguintes, o bloco é ampliado e seu conteúdo preservado, mesmo que o sistema o mova para outro endereço.

O resultado vai para um ponteiro temporário (`cont` ou `temp`) e só é atribuído ao ponteiro global depois de confirmado que não é `NULL`. Assim, se a memória acabar, o vetor antigo continua válido. Um cuidado decorrente: como `realloc` pode mover o bloco, um ponteiro obtido antes por `buscar_aluno` não deve ser usado depois de um novo cadastro.

Seção 6 – Arquivos Binários

O sistema usa um único arquivo binário, `dados_sistema.bin`, para guardar tudo o que foi cadastrado. Em arquivo binário, os bytes das variáveis são gravados exatamente como estão na memória, sem conversão para texto.

Estrutura do arquivo

Quais dados são armazenados. O arquivo guarda os dois contadores e depois os dois vetores completos: todos os alunos (com nome, RA, notas, disciplina embutida e média) e todas as disciplinas cadastradas.

Qual estrutura é gravada. O arquivo é uma sequência de quatro partes, sempre na mesma ordem:

Ordem	Conteúdo	Tipo gravado	Tamanho
1	Quantidade de alunos (`quant`)	`int`	1 inteiro
2	Quantidade de disciplinas (`quant_disciplinas`)	`int`	1 inteiro
3	Vetor de alunos	`Alunos`	`quant` structs
4	Vetor de disciplinas	`Disciplina`	`quant_disciplinas` structs

Os dois inteiros iniciais funcionam como cabeçalho: dizem ao programa, na leitura, quantas structs de cada tipo vêm a seguir. Sem eles, não haveria como saber onde termina o vetor de alunos e começa o de disciplinas. Cada struct é gravada de uma vez, com `sizeof` do tipo. Em geral, um `Alunos` ocupa 184 bytes e uma `Disciplina` 60 bytes, então um arquivo com 1 aluno e 1 disciplina tem cerca de 252 bytes. Esses valores dependem do compilador, que pode incluir bytes de alinhamento (padding) dentro das structs. O programa exibe o tamanho real ao salvar.

Quando ocorre o uso de fwrite, fread e fseek

- **`fopen` com `"wb"` e `"rb"`:** `salvar_dados` abre o arquivo para escrita binária, o que apaga o conteúdo anterior. `carregar_dados` abre para leitura binária. Se o arquivo não existir, como na primeira execução, `fopen` devolve `NULL` e a função apenas retorna, deixando o sistema começar vazio.
- **`fwrite`:** é chamada em `salvar_dados`, no encerramento do programa. Grava primeiro `quant` e `quant_disciplinas`, depois o vetor `alunos` inteiro e o vetor `disciplinas` inteiro. Os vetores só são gravados se tiverem ao menos um elemento.
- **`fread`:** é chamada em `carregar_dados`, na inicialização. Lê os dois contadores, usa-os para alocar os vetores com `malloc` e lê o conteúdo dos vetores na mesma ordem em que foi gravado.
- **`fseek` e `ftell`:** em `salvar_dados`, depois de gravar, `fseek(arq, 0, SEEK_END)` posiciona o cursor no fim do arquivo e `ftell` informa a posição, ou seja, o tamanho total em bytes, que é exibido ao usuário como confirmação. Aqui o `fseek` serve para medir o arquivo, e a leitura segue sempre do início ao fim.
- **`fclose`:** fecha o arquivo nas duas funções, o que garante que os dados sejam de fato descarregados no disco.

Persistência

Como os dados permanecem disponíveis após o encerramento do programa? Variáveis e vetores vivem na memória RAM, que é apagada quando o programa termina. Ao escolher "Sair", `main` chama `salvar_dados`, que copia os vetores da memória para `dados_sistema.bin` em disco. O arquivo continua existindo depois que o programa fecha. Na próxima execução, `main` chama `carregar_dados` antes de abrir o menu, que lê o arquivo, recria os vetores na memória e devolve o sistema ao estado em que foi deixado. O ciclo é: carregar, usar, salvar.

Limitações conhecidas

- Os dados só são gravados ao sair pelo menu. Se o programa fechar de forma abrupta, o que foi cadastrado na sessão se perde.
- O arquivo grava a struct bruta, então só é garantido que funcione no mesmo compilador e na mesma plataforma em que foi criado.
- Uma melhoria possível é conferir o valor devolvido por `fread` e `fwrite` para detectar arquivo corrompido ou gravação incompleta.

Bateria de Testes

Casos normais

Caso	Entrada	Resultado esperado
N1 – Cadastro válido de disciplina	Menu 2, opção 1: nome `Algoritmos`, código `101`, carga horária `80`	Mensagem "Disciplina cadastrada com sucesso!"; a disciplina aparece na listagem
N2 – Cadastro válido de aluno aprovado	Menu 1, opção 1: nome `Maria`, RA `1001`, disciplina `Algoritmos`, código `101`, carga `80`, notas 7, 8, 6, 7	Média 7,00, status Aprovado e mensagem de sucesso
N3 – Consulta de aluno pelo RA	Menu 1, opção 3, RA `1001`	Exibe nome Maria, RA 1001, média 7,00 e status Aprovado
N4 – Atualização de nota	Menu 1, opção 4, RA `1001`, nota `3`, nova nota `10`	Nota atualizada e média recalculada para 8,00
N5 – Persistência entre execuções	Cadastrar aluno e disciplina, sair (opção 0 nos menus) e abrir o programa de novo	Mensagem com o tamanho do arquivo ao sair; ao reabrir, os registros aparecem nas listagens

Casos limite

Caso	Entrada	Resultado esperado
L1 – Média exatamente no limite	Aluno com notas 5, 5, 5, 5	Média 5,00 e status Aprovado (a regra é média maior ou igual a 5,0)
L2 – Logo abaixo do limite	Aluno com notas 5, 5, 5, 4	Média 4,75 e status Reprovado
L3 – Primeiro cadastro	Primeiro aluno após iniciar	`realloc` a partir de `NULL` aloca o vetor e o

Caso	Entrada	Resultado esperado

com vetor vazio	sem dados	cadastro é concluído
L4 – Vetor expandido	Cadastrar um segundo e um terceiro aluno em seguida	Vetor realocado a cada cadastro; os registros anteriores continuam intactos na listagem

Casos inválidos

Caso	Entrada	Resultado esperado
I1 – Código de disciplina inexistente	Menu 2, opção 3, código `9999`	"Disciplina nao encontrada!"
I2 – RA inexistente	Menu 1, opção 3, RA `9999`	"Aluno nao encontrado!"
I3 – Código de disciplina duplicado	Cadastrar disciplina com código `101` já existente	"Ja existe uma disciplina com esse codigo!" e nenhum novo registro
I4 – Número de nota fora do intervalo	Menu 1, opção 4, RA válido, nota a alterar `7`	"Nota invalida! Escolha de 1 a 4." e nenhuma nota é alterada
I5 – Opção de menu inexistente	Qualquer menu, opção `9`	"Opcao invalida!" e o menu é exibido de novo

Observação sobre limitações encontradas nos testes

O código atual não valida alguns dados, o que o grupo pode registrar como melhoria futura: RA repetido é aceito, notas fora do intervalo de 0 a 10 são aceitas e o `scanf("%s")` não lê nomes com espaço. Digitar letras onde se espera número também não é tratado e pode prender o menu em repetição.

Reflexão Individual

Integrante: Frederico de Medeiros Freitas – RA 263447

1. Qual parte do sistema você mais contribuiu para implementar?

Estrutura inicial do código, menu (cases), bateria de testes e Persistência em arquivo binário (salvar_dados, carregar_dados).

2. Qual conceito foi mais difícil?

O conceito mais complicado foi a utilização de 2 ponteiros na função de modificar notas diretamente, onde é uma utilização que ainda possuo dificuldade. Além disso, pesquisar sobre arquivos e entender cada tipo também teve uma dificuldade, pois é um conceito novo que precisou ser aprendido.

3. O que você aprendeu sobre ponteiros?

Aprendi que ponteiros armazenam endereços de memória e permitem acessar ou modificar valores indiretamente. Reforcei também que o ponteiro duplo (**) permite modificar o próprio ponteiro da função que o chamou, sendo útil em situações como a realocação de memória com realloc e a atualização de vetores dinâmicos.

4. O que você aprendeu sobre arquivos binários?

Aprendi que arquivos binários permitem armazenar e recuperar dados de forma eficiente utilizando fwrite e fread, inclusive estruturas (struct) inteiras. Também compreendi que os contadores gravados antes dos vetores indicam a quantidade de elementos que devem ser lidos, enquanto fseek e ftell permitem manipular posições e verificar o tamanho dos arquivos. Por fim, entendi que, para garantir a persistência dos dados, é necessário salvá-los corretamente antes de encerrar o programa.

5. Como a IA ajudou e quais limitações ela apresentou?

A IA ajudou principalmente na criação e na compreensão de partes do código em C, como o uso de vetores dinâmicos com realloc, as funções de busca, os ponteiros para ponteiros e a gravação e leitura de arquivos binários. Também contribuiu para organizar o relatório técnico e elaborar os casos de teste. Algumas sugestões precisaram ser adaptadas para se integrar corretamente ao código do grupo, como a organização das estruturas, dos menus e das funções. Além disso, foi necessário conferir as explicações e testar as implementações do código na prática.

Integrante: Gabriel José Vieira Leme Teles – RA 263429

1. **Qual parte do sistema você mais contribuiu para implementar?**
   Contribuí principalmente com as funções de cadastro, listagem e busca de disciplinas. Trabalhei na implementação dessas funcionalidades para permitir registrar novas disciplinas, exibir as disciplinas cadastradas e buscar informações pelo código.
2. **Qual conceito foi mais difícil?**
   O conceito mais difícil foi trabalhar com ponteiros e alocação dinâmica de memória, principalmente entender como utilizar malloc, realloc e ponteiros para acessar e modificar os dados.
3. **O que você aprendeu sobre ponteiros?**
   Aprendi que os ponteiros armazenam endereços de memória e permitem acessar e modificar dados. No projeto, eles foram utilizados para manipular os vetores dinâmicos e buscar disciplinas pelo código.
4. **O que você aprendeu sobre arquivos binários?**
   Aprendi que os arquivos binários permitem salvar os dados do sistema e carregá-los novamente quando o programa é executado. Utilizamos fwrite para salvar as informações e fread para carregá-las, mantendo os dados mesmo depois de fechar o programa.
5. **Como a IA ajudou e quais limitações ela apresentou?**

   A IA ajudou a entender os conceitos de programação em C, identificar erros e sugerir soluções para o código. Porém, foi necessário analisar e testar as sugestões, pois nem sempre eram totalmente adequadas ao projeto ou fáceis de compreender sem estudar o funcionamento do código.

Integrante: Luis Fernando Martins Elias – RA 263253

1. Qual parte do sistema você mais contribuiu para implementar?

Eu contruibuí em como seria a estrutura do nosso projeto, o switch, quais as funcionalidades teria o nosso código, e em algumas funções nas quais são as as listagens de disciplinas e alunos e buscar alunos pelo RA e buscar disciplina pelo código.

2. Qual conceito foi mais difícil?

O conceito mais difícil foi trabalhar com ponteiros para ponteiros (Alunos **) e alocação dinâmica. Entender a lógica de passar o endereço do ponteiro por referência para conseguir redimensionar os vetores com o realloc e alterar dados estruturados sem perder a referência original deu bastante trabalho no começo.

3. O que você aprendeu sobre ponteiros?

Compreendi que os ponteiros são fundamentais para lidar com vetores dinâmicos, permitindo alocar e redimensionar o espaço de armazenamento sob demanda conforme novos cadastros são feitos.

4. O que você aprendeu sobre arquivos binários?

Aprendi a usar fwrite e fread para salvar o cadastro direto no disco e carregar tudo de volta quando o programa é aberto, garantindo que os dados fiquem salvos mesmo após fechar o sistema, e que usar funções como fseek e ftell foi fundamental para conferir o tamanho do arquivo e evitar leituras corrompidas. Ficou claro que salvar tudo certo antes de encerrar o sistema é o que garante que nenhum dado seja perdido.

5. Como a IA ajudou e quais limitações ela apresentou?

A IA ajudou bastante a tirar dúvidas de sintaxe, revisar o código, explicar conceitos mais complexos como o realloc, e apontar onde estavam alguns bugs chatos (como o loop do scanf). A limitação é que nem sempre a solução vinha pronta para o nosso código, exigindo que a gente testasse, adaptasse e entendesse bem a lógica para funcionar direito.